Lightning Warning Box

A low-cost, offline shelter countdown and adaptive all-clear for outdoor workers.

FOSSEE OSHW Make-A-Thon 2026
Theme: Lightning Detection
Author: Ananya Makkar
Date: 10 October 2026

1. Summary

Government sources cite roughly 2,000 to 2,500 lightning deaths a year in India. Outdoor workers such as farmers are exposed in open fields, and the usual guidance is to wait a fixed 30 minutes after the last nearby strike before going back out. This project designs a small device that uses only lightning-strike reports (time and estimated distance) to answer two questions: how long until the storm reaches me (shelter countdown) and when is it safe to leave (an adaptive all-clear instead of the fixed wait). It shows GREEN, YELLOW or RED on three LEDs and sounds a buzzer during RED. Main findings, all from simulation: (a) within about 10 minutes of arrival the countdown has a median absolute error of about 3 minutes; (b) on smooth simulated storms an adaptive all-clear saves about 14.5 minutes at a somewhat higher unsafe rate than the fixed rule (10.8% vs 7.4%); (c) the same rule fails on bursty storms (64.9% unsafe) because it mistakes a lull for the end of the storm; (d) a rule that tracks the storm's path held up on bursty storms (11.0% unsafe vs 8.8% for the fixed rule, saving 15.3 minutes).

2. Problem statement and objectives

- Estimate the time until a storm is within 10 km using nothing but strike reports from a low-cost sensor.
- Decide when it is safe to leave without always waiting a fixed 30 minutes, and measure what that costs in safety.
- Give a simple three-state indication (GREEN, YELLOW, RED) plus a buzzer that needs no phone or internet.
- Keep the design cheap, offline and fully open (schematic, firmware, simulation code and results).

3. System overview

An AS3935 lightning-sensor module reports each detected strike with an estimated distance. An ESP32 keeps a history of reports, runs the countdown and all-clear logic, and drives three LEDs and a buzzer. The sensor is connected over SPI; strike reports are read when the sensor raises its interrupt (IRQ) line.

4. Methods

4.1 Storm and sensor simulation

A storm cell moves in a straight line past the device at constant speed. Strikes arrive as a Poisson process whose rate follows a bell curve around the moment of closest approach, and each strike lands near the storm centre with a scatter of about 2 km. The sensor model then adds three imperfections: 20% of strikes are missed, distance error grows with distance (about 10%), and false triggers occur at random (about 0.05 per minute, at random distances up to 40 km). Reports are limited to 40 km, in line with the roughly 40 km range commonly quoted for the AS3935

Bursty storms are a harder test set: each storm alternates between active spells (mean 6 minutes) and quiet spells (mean 8 minutes, at 5% of the normal rate), with the long-run average rate kept the same. The burst settings are my assumptions, chosen to be difficult, not measured values.

4.2 Shelter countdown

The device fits a straight line through the strike distances from the last 20 minutes, drops outliers (false triggers) and refits, then extrapolates to a 10 km threat distance. It gives no estimate if fewer than 6 strikes are available or the storm is not clearly approaching.

4.3 All-clear rules compared

All rules use strikes reported within 16 km (about 10 miles) and are compared with the fixed rule (always wait 30 minutes after the last close strike):-
Mean-gap rule: wait k times the mean gap between recent close strikes, clipped to 5-30 minutes.
Max-gap rule: wait m times the longest gap seen so far in the storm, clipped to 5-30 minutes.- Path rule: fit the storm's distance trend and wait until the projected distance passes a margin (22 or 26 km). If the storm is not clearly moving away, wait the full 30 minutes.

4.4 Alert logic (GREEN / YELLOW / RED)- RED: two strikes within 10 km in the last 5 minutes, or a countdown of 10 minutes or less (ignored if strikes are clearly receding). RED is held for at least 10 minutes to prevent flicker.- YELLOW: strikes within 30 km before the storm gets close, or still inside the all-clear wait after a close strike.- GREEN: otherwise. The Python reference implementation and the firmware use the mean-gap rule with k = 12. See Section 7 for why this matters.

4.5 Evaluation protocol:

A storm is threatening if a real strike came within 16 km and the sensor reported at least one close strike.
Unsafe: a real strike within 16 km occurs after the device is declared all-clear (first GREEN).
Saved: the fixed rule's all-clear time minus the tested rule's, averaged over storms.
Settings were tuned on 200 storms (seeds 0-199) and the final comparison was run on 500 fresh smooth and 499 fresh bursty storms never used during tuning. All scripts are seeded and reproducible.

5. Hardware design

The schematic was drawn in KiCad 10. It shows the bare ESP32-WROOM-32 module symbol; in practice a development board would be used, which typically supplies the EN pull-up, USB and 3.3 V regulation

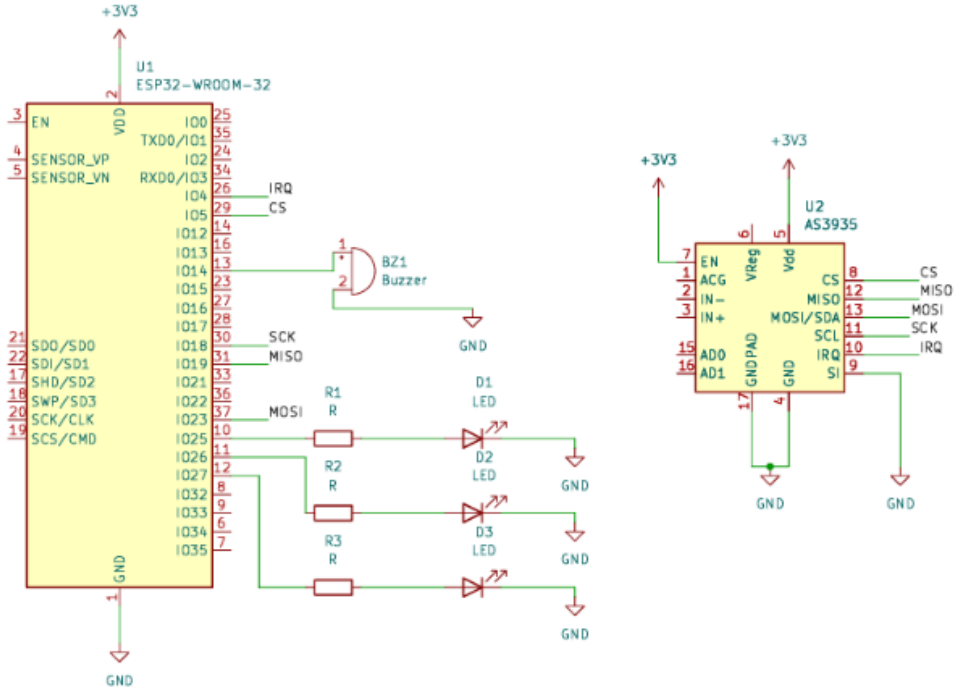


Figure 1. KiCad schematic: ESP32 (U1), three LED indicator chains, buzzer (BZ1) and the AS3935 sensor (U2), connected to the ESP32 by net labels (SCK, MISO, MOSI, CS, IRQ). The schematic symbols are colourless and the resistors show the default value R; the Wokwi build uses green, yellow and red LEDs and 220 ohm resistors.

Function	ESP32 pin	Connection

Green LED	IO25	Series 220 ohm resistor, LED, GND
Yellow LED	IO26	Series 220 ohm resistor, LED, GND
Red LED	IO27	Series 220 ohm resistor, LED, GND
Buzzer	IO14	Buzzer to GND (sounds during RED)
AS3935 SCK / MISO / MOSI / CS	IO18 / IO19 / IO23 / IO5	SPI to the sensor
AS3935 IRQ	IO4	Interrupt on strike detection
Power	VDD	+3.3 V; sensor Vdd and EN also on +3.3 V; sensor SI to GND (SPI mode); GND and GNDPAD to GND

Bill of materials

Item	**Qty**	**Notes**
ESP32 development board (ESP32-WROOM-32)	1	Arduino-compatible
AS3935 lightning-sensor breakout module	1	Pin order differs between vendors; check before wiring
LEDs: green, yellow, red	1 each	LED current limiting
Resistor, 220 ohm	3	Driven directly from IO14
Buzzer	1	Driven directly from IO14
Breadboard, jumper wires, USB cable or 5 V supply	-	

Firmware and simulation

The firmware (Arduino C++) ports the same countdown, all-clear and alert logic to the ESP32 and runs in the Wokwi simulator. Wokwi does not provide a lightning-sensor model that I used, so the firmware does not read a sensor: it replays a stored strike list (one simulated storm exported from the Python simulator) through the alert logic, with one simulated minute lasting 0.5 s, and lights the LEDs and buzzer accordingly. Wokwi is free but not open source; the firmware, circuit description and schematic are open. Verification status (partial). For this storm the Python code produces state changes at minutes 0 (GREEN), 14 (YELLOW), 24 (RED), 61 (YELLOW) and 74 (GREEN). In the simulator I saw the green LED lit near the end of a fast replay (Python predicts GREEN from minute 74) and the red LED lit about 18 s into a replay at 0.5 s per minute (Python predicts RED from about 12 s to 30 s). I did not time every transition and could not display the Serial Monitor, so the firmware is not confirmed to reproduce the full Python sequence. 6. Results All numbers are from simulated storms. Unsafe and saved are defined in Section 4.5. 6.1 Shelter countdown (100 smooth storms)

Minutes before arrival	**Storms**	**No estimate**	**Median Error**	**Median absolute error**
30	53	41	+1.8 min	15.0 min
20	58	20	-4.5 min	8.6 min
10	59	6	-2.1 min	3.6 min
5	59	4	-1.7 min	3.0 min

The countdown is useful within about 10 minutes of arrival, with a median error of about 3 minutes on the early (safe) side. Far out it mostly gives no estimate because too few strikes have arrived. A small number of estimates were dangerously wrong, saying there was plenty of time when the storm was nearly overhead

6.2 Adaptive all-clear: safety versus time saved

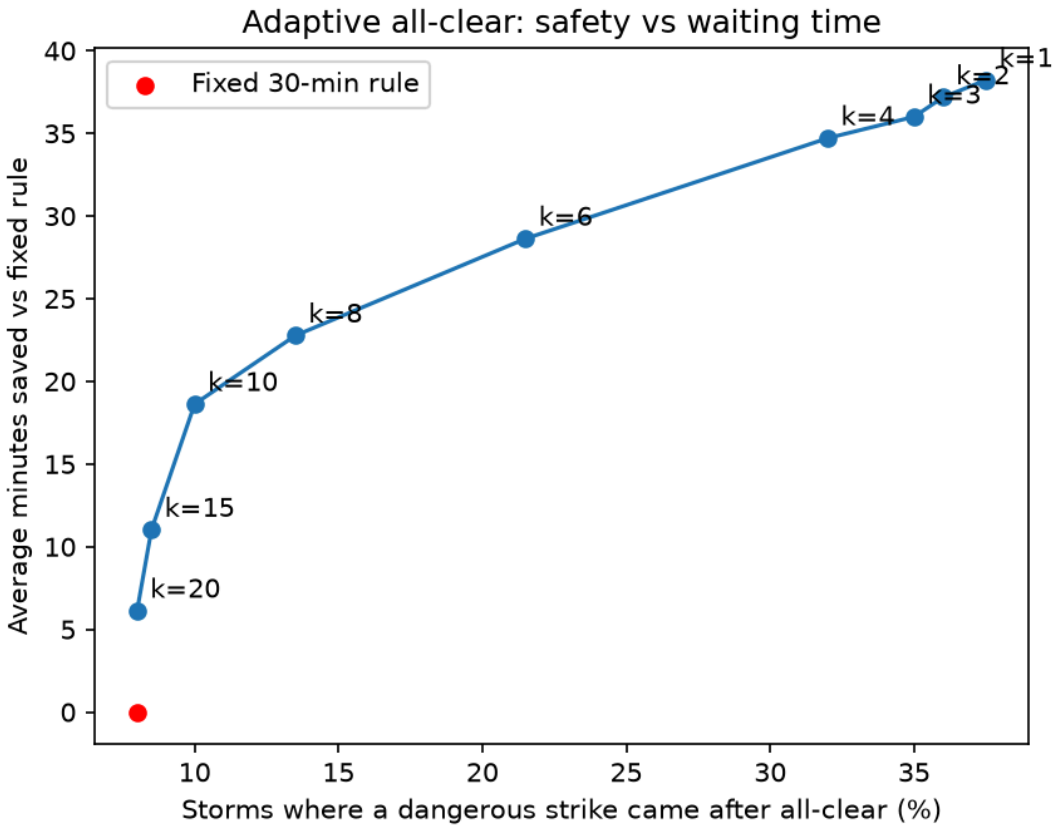


Figure 2. Mean-gap rule on 200 smooth storms (tuning set). Each point is a setting of k; the red dot is the fixed 30-minute rule (8.0% unsafe). Larger k is more cautious: k = 20 saves 6.1 min at 8.0% unsafe, k = 15 saves 11.0 min at 8.5%, k = 10 saves 18.6 min at 10.0%, k = 6 saves 28.6 min at 21.5%.

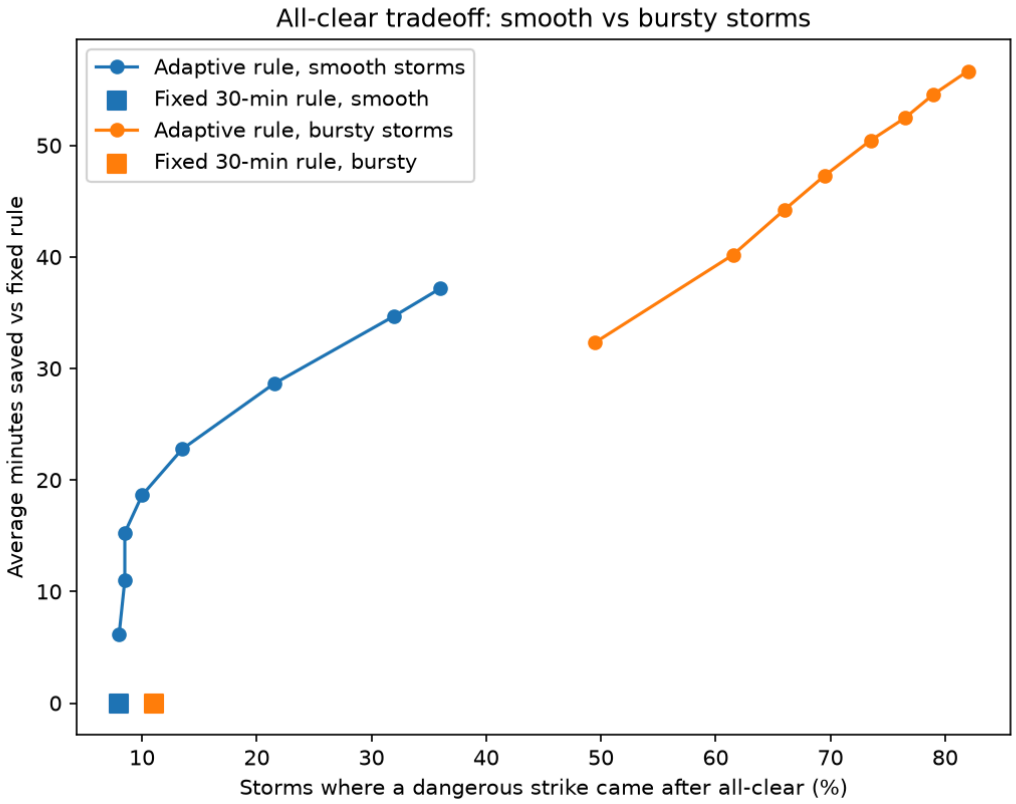


Figure 3. The same rule on smooth (blue) and bursty (orange) storms. On bursty storms every setting is far less safe, and the large time savings are not real: the rule clears during lulls.

6.3 Final comparison on fresh storms (holdout)

Rule	**Smooth: unsafe/unsaved**	**Bursty: unsafe/saved**
Fixed 30-minute rule 10.8% / 14.5 min	7.4% / 0	8.8% / 0
Mean-gap, k = 12 (used in the firmware)	8.8% / 10.0 min	64.9% / 45.0 min
Max-gap, m = 3	8.8% / 10.0 min	29.9% / 18.4 min
Path rule, margin 22 km	9.6% / 17.5 min	11.0% / 15.3 min
Path rule, margin 26 km	8.6% / 11.4 min	10.2% / 10.2 min

- On smooth storms the mean-gap rule saves about 14.5 minutes at a modestly higher unsafe rate. Its earlier tuning set result (8.5% unsafe) was optimistic
- On bursty storms the mean-gap rule fails: it clears mid-storm in 56.7% of storms against 0.6% for the fixed rule.

- The path rule held up on bursty storms (11.0% unsafe, 15.3 minutes saved) because it tracks where the storm is going, not only how often it strikes. It is still slightly less safe than the fixed rule, so the honest claim is close to the fixed rule's safety with 10-17 minutes saved, not as safe as it.

6.4 Full alert system (mean-gap rule, 200 smooth storms)

	Unsafe	Safe vs fixed rule
Fixed 30-minute rule	8.0 %	0
Full alert system (GREEN/YELLOW/RED)	7.5 %	13.7 min mean (12.1 median)

The 0.5-point difference is one storm in 200 and is not evidence of improved safety.

	Safe	Cleared before the real storm	Cleared mid storm
Fixed 30-minute rule	92.0%	7.0 %	1.0 %
Mean-gap, k = 12	91.5%	7.0 %	1.5 %

Most unsafe cases are the device clearing before the real storm arrives. I did not trace each one, but my explanation is a false trigger within 16 km starting the all-clear clock while the real storm is still far away (in 82 of 200 storms the first close strike the sensor reported was a false one). Clearing during the storm is rare on smooth storms.

7. Limitations

- Everything is simulated. Storm path, strike process, burst timing and false-trigger rate are my assumptions, not measurements. The burst parameters in particular could change the results a lot.
- Straight-line storm motion flatters the path rule. Real storms change speed and direction.
- The firmware uses the weaker rule. The mean-gap rule (k = 12) runs on the device and in the Python reference. The path rule performs better on bursty storms but is not ported to the firmware.
- Re-alerting. In 70 of 200 smooth storms the device went back on alert after its first GREEN, mostly because of a single false trigger. A confirmation filter reduced this only slightly (to 56) while raising unsafe cases to 13.5%, so it was reverted.

- Countdown tail risk: a few estimates were off by an hour or more in the unsafe direction
- Tuning: k = 12 and the path margins were chosen by looking at results on the tuning storms; only the final comparison used fresh storms.
- Hardware not verified. The AS3935 pin functions used (SI to ground for SPI, EN to Vdd, grounded exposed pad) were set from memory and not confirmed against the datasheet. The schematic uses the bare chip symbol, so the antenna and regulator pins (ACG, IN-, IN+, VReg, AD0, AD1) are open; a breakout module provides them. KiCad's electrical rules check also reports the unused ESP32 pins, which are intentional. The firmware replay was only partly verified (Section 5).
- Safety. This is a student project and must not be used as a real safety device.

8. Prior work

This project does not claim to invent lightning detection. Existing work includes IITM Pune's Damini lightning-alert app, built on a national lightning-location sensor network, which sends GPS-based alerts when lightning occurs within 20 km and 40 km of the user [1, 2]; the volunteer Blitzortung.org network of community-run VLF receivers that locate strikes from time of arrival [3]; and projects built around the AS3935 lightning-sensor chip. Commercial lightning detectors also exist; I have not surveyed them. I have not searched the literature on adaptive all-clear rules, so I make no novelty claim for them. What this project adds is an open, reproducible evaluation of all-clear rules against the fixed 30-minute rule, including the failure on bursty storms, and an offline device design aimed at users without a smartphone or signal.

9. Open source and reproducibility

Everything is released openly in the submission folder: Python simulation and evaluation code (src), the KiCad project and schematic (hardware), the Wokwi firmware and circuit files (wokwi), result plots and a README. Licences: code under the MIT licence (LICENSE file included); documentation and hardware design files under CC BY-SA 4.0. To reproduce the results (Python 3 with numpy and matplotlib), run from the repository root: python src/evaluate_countdown.py, src/tradeoff.py, src/tradeoff_bursty.py, src/tradeoff_v2.py, src/tradeoff_v3.py, src/holdout.py and src/evaluate_alert.py. All scripts are seeded, so the printed numbers should match the tables above. The diagnose scripts are debugging tools; they found an evaluation mistake early on (measuring the last GREEN instead of the first GREEN gave a false 0% unsafe result).

10. References

[01] IITM Pune / Ministry of Earth Sciences, Damini mobile app description (Vikaspedia): en.vikaspedia.in/social-welfare/disaster-management-1/damini-mobile-app

[2] Ministry of Earth Sciences, Lok Sabha unstarred question on deaths due to lightning strikes (2022): moes.gov.in/sites/default/files/LS-in-English-5533-06042022.pdf
[3] Blitzortung.org, project description: blitzortung.org/cover_your_area.php